

Introduction to Blockchain and Smart Contracts

Lecture 10: Smart Contract Vulnerabilities

Subodh Sharma | IIT Delhi | March 08, 2026



Reentrancy

- Sender sends ether to an external address before updating its own state
- Attacker calls withdraw
 - In fallback, attacker again invokes **withdraw**
- Fix: **Checks-effect-interactions**

```
contract Bank {
    mapping(address => uint) public balance;

    function deposit() external payable {
        balance[msg.sender] += msg.value;
    }

    function withdraw(uint amount) external {
        require(balance[msg.sender] >= amount);

        // External call first
        (bool ok, ) = msg.sender.call{value: amount}("");
        require(ok);

        // State update later
        balance[msg.sender] -= amount;
    }
}
```

Reentrancy

Attacker

- Exploits the semantics of empty calldata in the external call of the bank contract
 - `receive()` is invoked as a side-effect

```
contract ReentrancyAttacker {
    IBadBank public bank;

    constructor(address _bank) {
        bank = IBadBank(_bank);
    }

    function attack() external payable {
        bank.deposit{value: msg.value}();
        bank.withdraw(msg.value);
    }

    receive() external payable {
        if (address(bank).balance >= 1 ether) {
            bank.withdraw(1 ether);
        }
    }
}
```


Access Control

- Critical function callable by anyone
 - Authorisation check is wrong
- Anyone can call **drain** and the siphon off the whole balance
- Fix: access modifier!

```
contract Vault {  
    address public owner;  
  
    constructor() {  
        owner = msg.sender;  
    }  
  
    function drain() external {  
        payable(msg.sender).transfer(address(this).balance);  
    }  
}
```

Access Control

- Critical function callable by anyone
 - Authorisation check is wrong
- Anyone can call **drain** and the siphon off the whole balance
- Fix: access modifier!

```
modifier onlyOwner() {  
    require(msg.sender == owner, "not owner");  
    _;  
}  
  
function withdrawAll() external onlyOwner {  
    payable(owner).transfer(address(this).balance);  
}
```

```
function drain() external {  
    require(msg.sender == owner, "not owner");  
    payable(owner).transfer(address(this).balance);  
}
```


Integer Underflow/Overflow + Unchecked Calls

- Arithmetic wraps around (dangerous in older Solidity versions)
 - FIX:
 - move to Solidity 0.8; all overflows/underflows lead to **revert**
 - Use **safemath** libraries
- Unchecked External Call Return Values
 - Ether is not sent but the reward is set to 0
 - FIX: Check the return value with **require**

```
function mint() external {  
    balances[msg.sender] += 250;  
}
```

```
contract Payout {  
    mapping(address => uint) public rewards;  
  
    function claim() external {  
        uint amount = rewards[msg.sender];  
        rewards[msg.sender] = 0;  
  
        msg.sender.call{value: amount}("");  
  
        (bool ok, ) = msg.sender.call{value: amount}("");  
        require(ok, "transfer failed");  
    }  
}
```

Denial of Service via Unexpected Revert

- Single malicious user can revert **block the entire txn**
- **Fix:** Use pull payment model
 - Let each user claim their funds individually

```
contract Auction {
    address[] public bidders;
    mapping(address => uint) public refunds;

    function refundAll() external {
        for (uint i = 0; i < bidders.length; i++) {
            address bidder = bidders[i];
            payable(bidder).transfer(refunds[bidder]);
            refunds[bidder] = 0;
        }
    }
}
```

Attacker



```
receive() external payable {
    revert("no thanks");
}
```

Denial of Service via Unexpected Revert

- Single malicious user can revert **block the entire txn**
- **Fix:** Use pull payment model
 - Let each user claim their funds individually

```
contract GoodRefund {
    mapping(address => uint256) public refundOf;

    function claimRefund() external {
        uint256 amount = refundOf[msg.sender];
        require(amount > 0, "nothing to claim");

        // Effects first
        refundOf[msg.sender] = 0;

        // Interaction last
        (bool ok, ) = payable(msg.sender).call{value: amount}("");
        require(ok, "refund failed");
    }

    receive() external payable {}
}
```


Front Running (Txn Ordering Dependence)

- A malicious miner can see a pending transaction in the mempool and submit another txn with higher gas
- **Not a syntax bug**
- **Fix:** Commit-reveal schemes or sealed bidding

```
contract BadNameRegistry {  
    mapping(bytes32 => address) public ownerOf;  
  
    function register(string calldata name) external {  
        bytes32 h = keccak256(bytes(name));  
        require(ownerOf[h] == address(0), "taken");  
        ownerOf[h] = msg.sender;  
    }  
}
```

Front Running (Txn Ordering Dependence)

Solutions

- **Commit phase:**
 - User submits only a hash of (sender, name, secret)
- **Reveal phase:**
 - User later reveals (name, secret)
- **Another solution: Sealed/Blind bidding scheme.**
(READING Assignment)

```
function commit(bytes32 commitment) external {
    require(block.timestamp < commitDeadline, "commit phase over");
    require(commitmentOf[msg.sender] == bytes32(0), "already committed");
    commitmentOf[msg.sender] = commitment;
}

function reveal(string calldata name, bytes32 secret) external {
    require(block.timestamp >= commitDeadline, "reveal not started");
    require(block.timestamp < revealDeadline, "reveal phase over");

    bytes32 expected = keccak256(abi.encode(msg.sender, name, secret));
    require(commitmentOf[msg.sender] == expected, "bad reveal");

    bytes32 nameHash = keccak256(bytes(name));
    require(ownerOf[nameHash] == address(0), "name taken");

    ownerOf[nameHash] = msg.sender;
    delete commitmentOf[msg.sender];
}
```

Oracle Price Manipulation

- **Setting:** An exchange such as Uniswap holding two reserves
 - Say TKN and USDC
 - Trade changes the reserve balances and therefore change the pool-implied price
 - A vulnerable oracle says: the price of TKN is whatever this DEX pool says right now
 - Lending protocol: Lets users deposit TKN as collateral and borrow USDC against it.
 - Attacker: uses an uncollateralized borrow (that must be returned with a fee) in the same transaction and exploits it

Oracle Price Manipulation

Attack Scenario

- **DEX Pool** has 1000 TKN and 1million USDC; **spot price 1 TKN = 1000 USDC**
- Suppose attacker borrows 0.4 million USDC in a flash loan
 - The attacker uses that capital in the same transaction by buying TKN with USDC available on the pool. This changes the reserve ratio
 - The pool goes from (1000, 1million) to (714, 1.4 million)
 - The spot price inflates to 1 TKN = 1960 USDC
 - This manipulated spot price of TKN is exploited to attackers advantage (by overestimating a collateral value that the attacker holds)
- **FIX: Do not use Spot pricing for lending, liquidation, minting or collateral valuation**